Question Paper

Snowflake Data Cloud — Core Concepts, Loading, Transformation & Apps

Section	Topic	No. of Questions	Marks
EASY	Data Loading / Time Travel / Data Sharing	1	10
EASY	Cortex Analyst (Semantic View)	1	10
MEDIUM	Snowpipe + Dynamic Table (Aggregation)	1	20
MEDIUM	Snowpipe + Dynamic Table SCD Type 1	1	20
MEDIUM	Snowpipe + Dynamic Table SCD Type 2	1	20
MEDIUM	Snowpark + Streamlit (Food Orders)	1	20
MEDIUM	Snowpark + Streamlit (Movie Bookings)	1	20
MEDIUM	Snowpark + Streamlit (Ride Trips)	1	20
EXTRA	Semi-Structured Data (JSON)	1	Bonus
EXTRA	RBAC, Masking & Row Access Policy	1	Bonus
	TOTAL CORE QUESTIONS	**8**	**140**
	TOTAL WITH EXTRA	**10**	**—**

★ *Dataset files for each question are provided in the corresponding zip folders.*

★ *Extra questions include their own datasets within this document.*

★ *Submit all required screenshots as specified in each question's Submission Requirements.*

■■ SECTION: EASY ■■

QUESTION 1 – EASY – DATA LOADING, UNLOADING, TIME TRAVEL & DATA SHARING

Marks: 10	Difficulty: Easy

Objective

Load store inventory data from an AWS S3 external stage, perform DML operations, recover deleted data using Snowflake Time Travel, unload filtered records to an internal stage, and share the final table using Secure Data Sharing.

Dataset File: store_inventory.csv (20 rows)

Dataset Schema

The file contains the following columns:

- PRODUCT_ID, PRODUCT_NAME, CATEGORY, BRAND, WAREHOUSE, STOCK_QTY, REORDER_LEVEL, UNIT_COST, SELLING_PRICE

Dataset Preview (first 5 rows)

RODUCT_ID	PRODUCT_NAME	CATEGORY	BRAND	WAREHOUSE	STOCK_QTY	REORDER_LEVEL	UNIT_COST	ELLING_PRICE
P001	Wireless Mouse	Accessories	LogiTech	WH-BLR	45	20	650.00	999.00
P002	Mechanical Keyboard	Accessories	KeyPro	WH-BLR	28	15	1800.00	2799.00
P003	USB-C Hub	Accessories	ConnectX	WH-MUM	32	12	1200.00	1899.00
P004	Laptop Stand	Accessories	ErgoDesk	WH-MUM	18	10	900.00	1499.00
P005	27-inch Monitor	Electronics	ViewMax	WH-DEL	14	8	12500.00	16999.00

Step 1 – Create Environment & Load Data

Create:

- Database: **STORE_INV_DB**
- Schema: **RAW**
- CSV File Format: **CSV_FMT** (TYPE=CSV, FIELD_DELIMITER=',', SKIP_HEADER=1)
- External Stage: **S3_STORE_STAGE** (using AWS S3 path and credentials provided)

Create the source table:

```
CREATE OR REPLACE TABLE STORE_INV_DB.RAW.STORE_INVENTORY ( PRODUCT_ID STRING, PRODUCT_NAME STRING, CATEGORY STRING, BRAND STRING, WAREHOUSE STRING, STOCK_QTY NUMBER, REORDER_LEVEL NUMBER, UNIT_COST NUMBER(10,2), SELLING_PRICE NUMBER(10,2) );
```

Load **store_inventory.csv** from S3_STORE_STAGE using COPY INTO.

✔ *Expected Row Count: 20*

Step 2 – DML & Time Travel

UPDATE: For PRODUCT_ID = 'P006', change STOCK_QTY to 0 and note the query ID.

DELETE: Delete the record where PRODUCT_ID = 'P017'. Capture the DELETE query ID.

Using Time Travel:

- Show the pre-update values for PRODUCT_ID = 'P006' using BEFORE (STATEMENT => '<update_query_id>')
- Restore the deleted record for PRODUCT_ID = 'P017' back into STORE_INVENTORY using its pre-delete state

✔ *Expected Final Row Count after restoration: 20*

Step 3 – Unload Low-Stock Products

Create an internal named stage: **UNLOAD_STAGE**

Unload only records where STOCK_QTY <= REORDER_LEVEL into @UNLOAD_STAGE/low_stock/ using CSV format with a header.

Verify the unloaded file using: LIST @STORE_INV_DB.RAW.UNLOAD_STAGE;

✔ *Expected: Products P005, P009, P010, P011, P015, P016, P017 (after restoration), P018, P020 qualify*

Step 4 – Create Secure Data Share

Create: **INVENTORY_SHARE**

Grant: USAGE on STORE_INV_DB, USAGE on schema RAW, SELECT on STORE_INVENTORY

Add the consumer Snowflake account identifier provided by the trainer to the share.

Verify: SHOW GRANTS TO SHARE INVENTORY_SHARE;

Submission Requirements

1. Data Load & Time Travel Validation – STORE_INVENTORY with 20 rows, pre-update values for P006, and restored P017.
2. Data Unload Validation – Show UNLOAD_STAGE with the low-stock file.
3. Secure Share Validation – Show INVENTORY_SHARE with table privilege and consumer account.

Evaluation Rubric

#	Criterion	Marks
1	Environment, Data Load & Time Travel – Objects created, CSV loaded, DML performed, Time Travel used correctly	5
2	Data Unload – Low-stock records correctly unloaded to internal stage	2
3	Secure Data Sharing – INVENTORY_SHARE created with correct privileges and consumer account	3
	TOTAL	**10**

★ *Award marks based on correct final outputs. Partial marks allowed.*

QUESTION 2 – EASY – CORTEX ANALYST WITH SEMANTIC VIEW

Marks: 10	Difficulty: Easy

Objective

Build a hospital analytics star schema using the provided dataset, create a Semantic View with table relationships and key business metrics, and use Snowflake Cortex Analyst to answer natural-language business questions about hospital visits, doctors, and departments.

Dataset: Hospital_Cortex_Analyst (folder contains 4 CSV files)

Tables & Row Counts

Table	File	Rows	Key Columns
DIM_DEPARTMENT	dim_department.csv	5	DEPARTMENT_ID, DEPARTMENT_NAME, BLOCK
DIM_DOCTOR	dim_doctor.csv	7	DOCTOR_ID, DOCTOR_NAME, DEPARTMENT_ID, DESIGNATION, EX
DIM_PATIENT	dim_patient.csv	12	PATIENT_ID, PATIENT_NAME, GENDER, AGE_GROUP, CITY, INSUR
FACT_VISIT	fact_visit.csv	30	VISIT_ID, VISIT_DATE, PATIENT_ID, DOCTOR_ID, DEPARTMENT_ID CONSULTATION_FEE, TREATMENT_COST, DISCOUNT_AMT, STAY_

Step 1 – Create Environment & Tables

Create: Database **HOSPITAL_ANALYST_DB**, Schema **MART**, CSV Format **CSV_FMT**, Internal Stage **HOSP_STAGE**

Create and load all four tables above. Verify row counts after loading.

Step 2 – Create Semantic View: HOSPITAL_ANALYTICS_SV

Configure relationships:

- FACT_VISIT.DOCTOR_ID = DIM_DOCTOR.DOCTOR_ID
- FACT_VISIT.PATIENT_ID = DIM_PATIENT.PATIENT_ID
- FACT_VISIT.DEPARTMENT_ID = DIM_DEPARTMENT.DEPARTMENT_ID
- DIM_DOCTOR.DEPARTMENT_ID = DIM_DEPARTMENT.DEPARTMENT_ID

Include these dimensions: DEPARTMENT_NAME, BLOCK, DOCTOR_NAME, DESIGNATION, PATIENT_NAME, GENDER, AGE_GROUP, CITY, INSURANCE_PROVIDER, VISIT_TYPE, READMITTED, VISIT_DATE

Configure these facts: CONSULTATION_FEE, TREATMENT_COST, DISCOUNT_AMT, STAY_DAYS

Create these metrics:

- TOTAL_REVENUE = SUM(CONSULTATION_FEE + TREATMENT_COST - DISCOUNT_AMT)
- TOTAL_VISITS = COUNT(DISTINCT VISIT_ID)
- AVG_TREATMENT_COST = AVG(TREATMENT_COST)
- READMISSION_RATE = COUNT(CASE WHEN READMITTED='Y' THEN 1 END) / COUNT(*) * 100
- AVG_STAY_DAYS = AVG(STAY_DAYS) [for IPD visits only]

Step 3 – Cortex Analyst Testing

Add at least 1 verified query: Total revenue by department

Using the Cortex Analyst playground, ask:

1. Which doctor has the highest total revenue?
2. What is the average treatment cost by department?
3. Which city has the highest readmission rate?

Validate any one Cortex Analyst result using an equivalent SQL query.

Tricky Element:

Readmission Rate is a calculated metric involving conditional logic. Ensure the semantic model expresses this correctly. Also note that the STAY_DAYS metric for AVG must filter only IPD visits — pure OPD visits have STAY_DAYS = 0 and including them would skew the average.

Submission Requirements

1. Data Load Validation – All four tables with expected row counts.
2. Semantic View Validation – HOSPITAL_ANALYTICS_SV with relationships, metrics, and descriptions.
3. Cortex Analyst Output – Results for required questions and one SQL validation.

Evaluation Rubric

#	Criterion	Marks
1	Environment & Data Load – All tables created and loaded with correct row counts	3
2	Semantic View & Relationships – All 4 relationships and dimensions configured correctly	3
3	Business Metrics – TOTAL_REVENUE, TOTAL_VISITS, AVG_TREATMENT_COST, READMISSION_RATE, AVG_STAY_DAY	2
4	Cortex Analyst Testing & Validation – Required questions answered and one result validated via SQL	2
	TOTAL	**10**

■■ SECTION: MEDIUM ■■

QUESTION 3 – MEDIUM – SNOWPIPE + DYNAMIC TABLE AGGREGATION (IOT SENSOR)

Marks: 20	Difficulty: Medium

Objective

Build a continuous IoT sensor ingestion pipeline using Snowpipe from an AWS S3 external stage, ingest sensor readings in two batches, monitor pipe status, and create a Dynamic Table that aggregates sensor data by location and zone type for operational reporting.

Dataset Files: sensor_readings_file1.csv (15 rows) & sensor_readings_file2.csv (12 rows) — from IOT_Snowpipe folder

Dataset Preview

READING_ID	SENSOR_ID	LOCATION	ZONE_TYPE	READING_TS	TEMP_C	HUMIDITY_%	BATTERY_%	STATUS
1001	SEN-A01	Chennai Plant	Production	2026-10-12 08:00	31.20	62	92	NORMAL
1002	SEN-A02	Chennai Plant	Production	2026-10-12 08:02	32.10	64	88	NORMAL
1003	SEN-A03	Chennai Plant	Storage	2026-10-12 08:04	39.80	78	81	ALERT
1006	SEN-B03	Bengaluru Hub	Utility	2026-10-12 08:10	41.10	82	79	ALERT
1011	SEN-P02	Pune Facility	Storage	2026-10-12 08:20	38.90	80	73	ALERT

Step 1 – Create Environment

Create: Database **IOT_DB**, Schema **RAW**, CSV Format **CSV_FMT**, External Stage **S3_IOT_STAGE**

Create the target table:

```
CREATE OR REPLACE TABLE IOT_DB.RAW.SENSOR_READINGS ( READING_ID NUMBER, SENSOR_ID STRING,
LOCATION STRING, ZONE_TYPE STRING, READING_TS TIMESTAMP_NTZ, TEMPERATURE_C NUMBER(5,2),
HUMIDITY_PCT NUMBER, BATTERY_PCT NUMBER, STATUS STRING, FILE_NAME STRING, LOADED_AT
TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP() );
```

Step 2 – Create Snowpipe

Create: **SENSOR_PIPE** — configure with AUTO_INGEST=TRUE, populate FILE_NAME using METADATA$FILENAME.

Step 3 – Ingest Sensor Files

Load both batches through SENSOR_PIPE.

- After File 1: Expected 15 rows in SENSOR_READINGS
- After File 2: Expected 27 total rows

Verify grouped by FILE_NAME to confirm both files are represented.

Step 4 – Monitor Snowpipe

Check pipe status using SYSTEM$PIPE_STATUS. Verify recent ingestion using INFORMATION_SCHEMA.COPY_HISTORY.

Step 5 – Create Dynamic Table: DT_SENSOR_ZONE_SUMMARY

Create with TARGET_LAG = '1 minute'. For each LOCATION + ZONE_TYPE, calculate:

- TOTAL_READINGS — count of readings
- AVG_TEMPERATURE — average temperature
- MAX_TEMPERATURE — maximum temperature
- AVG_HUMIDITY — average humidity

- MIN_BATTERY — minimum battery level (critical for maintenance alerts)
- ALERT_COUNT — count of readings where STATUS = 'ALERT'

Refresh the Dynamic Table and verify it reflects all 27 sensor readings.

Pause the pipe after validation.

Tricky Element:

ALERT_COUNT requires conditional aggregation: COUNT(CASE WHEN STATUS='ALERT' THEN 1 END). MIN_BATTERY is critical for maintenance — sensors with BATTERY_PCT <= 75 need attention. Verify that ALERTs from Bengaluru Hub Utility (SEN-B03, SEN-B06) and Hyderabad Utility (SEN-H06) appear correctly in the summary.

Submission Requirements

1. Snowpipe Ingestion Validation – 27 rows, grouped by FILE_NAME.
2. Pipe Monitoring – Pipe status and COPY_HISTORY evidence.
3. Dynamic Table Output – DT_SENSOR_ZONE_SUMMARY with all required metrics.

Evaluation Rubric

#	Criterion	Marks
1	Environment & Snowpipe Ingestion – Stage, table, pipe created; both files ingested (27 rows)	7
2	Pipe Monitoring – Pipe status and COPY_HISTORY entries confirmed	5
3	Dynamic Table – DT_SENSOR_ZONE_SUMMARY aggregates correctly including ALERT_COUNT	6
4	Final Pipeline State – Pipe paused after validation	2
	TOTAL	**20**

QUESTION 4 – MEDIUM – SNOWPIPE + DYNAMIC TABLE WITH SCD TYPE 1 (EMPLOYEE)

Marks: 20	Difficulty: Medium

Objective

Load employee change records from an AWS S3 external stage using Snowpipe in multiple batches and use a Snowflake Dynamic Table to maintain SCD Type 1 — always reflecting only the latest version of each employee record based on UPDATED_AT.

Dataset Files: employee_batch1.csv (18), employee_batch2.csv (8), employee_batch3.csv (2) — from Employee_SCD_Dynamic folder

Dataset Preview — employee_batch1.csv (first 5 rows)

EMP_ID	EMP_NAME	DEPARTMENT	DESIGNATION	WORK_LOCATION	UPDATED_AT
6101	Aarav Sharma	Engineering	Software Engineer	Chennai	2026-09-01 09:00:00
6102	Meera Iyer	Engineering	Data Engineer	Bengaluru	2026-09-01 09:05:00
6103	Rohan Mehta	Finance	Finance Analyst	Mumbai	2026-09-01 09:10:00
6104	Kavya Rao	HR	HR Executive	Hyderabad	2026-09-01 09:15:00
6105	Arjun Nair	Sales	Sales Executive	Kochi	2026-09-01 09:20:00

Batch 2 contains 6 promotions/transfers + 2 new employees (6119, 6120). Batch 3 has 2 location changes.

Step 1 – Create Environment

Create: Database **EMPLOYEE_SCD_DB**, Schema **RAW**, CSV Format **CSV_FMT**, External Stage **S3_EMP_STAGE**

Create the raw table:

```
CREATE OR REPLACE TABLE EMPLOYEE_SCD_DB.RAW.EMPLOYEE_RAW ( EMP_ID NUMBER, EMP_NAME STRING,
DEPARTMENT STRING, DESIGNATION STRING, WORK_LOCATION STRING, UPDATED_AT TIMESTAMP_NTZ );
```

Step 2 – Create Snowpipe: EMPLOYEE_PIPE

Configure AUTO_INGEST=TRUE pointing to S3_EMP_STAGE loading into EMPLOYEE_RAW.

Step 3 – Ingest Batches & Create SCD Type 1 Dynamic Table

Load all three batches. Create **DT_EMPLOYEE_SCD1** with TARGET_LAG='1 minute'.

The Dynamic Table must show only the LATEST values per EMP_ID based on UPDATED_AT.

Required columns: EMP_ID, EMP_NAME, DEPARTMENT, DESIGNATION, WORK_LOCATION, LAST_UPDATED_AT

Expected row counts after all batches:

- EMPLOYEE_RAW = 28 rows | DT_EMPLOYEE_SCD1 = 20 unique employees

Step 4 – Validate Key Records

Verify EMP_ID 6102 (Meera Iyer): SCD1 must show DESIGNATION='Lead Data Engineer', WORK_LOCATION='Bengaluru' (from batch 3)

Verify EMP_ID 6108 (Priya Menon): SCD1 must show WORK_LOCATION='Mumbai' (changed in batch 3)

Verify EMP_ID 6119 and 6120 appear (added in batch 2).

Tricky Element:

The SCD Type 1 Dynamic Table uses ROW_NUMBER() OVER (PARTITION BY EMP_ID ORDER BY UPDATED_AT DESC) = 1 to retain only the latest record. Candidates must handle multiple updates to the same EMP_ID correctly, especially EMP_ID 6102 which appears in all three batches with progressively higher designations.

Submission Requirements

1. Snowpipe Ingestion – EMPLOYEE_RAW with 28 rows and FILE_NAME grouping.
2. SCD1 Final Output – DT_EMPLOYEE_SCD1 with 20 rows and correct latest values for 6102, 6108.
3. Dynamic Table Refresh Evidence – Refresh status confirmed.

Evaluation Rubric

#	Criterion	Marks
1	Environment & Snowpipe – Objects created, all batches ingested (28 rows)	6
2	SCD Type 1 Dynamic Table – DT_EMPLOYEE_SCD1 correctly shows latest record per employee (20 rows)	8
3	Key Record Validation – 6102 and 6108 show correct latest values	4
4	Dynamic Table Refresh & Pipeline State – Refresh confirmed, pipe paused	2
	TOTAL	**20**

QUESTION 5 – MEDIUM – SNOWPIPE + DYNAMIC TABLE WITH SCD TYPE 2 (PRODUCT)

Marks: 20	Difficulty: Medium

Objective

Load product catalogue change records from an AWS S3 external stage using Snowpipe in multiple batches and use a Snowflake Dynamic Table to maintain full SCD Type 2 history — tracking every version of each product with effective dates, current flag, and version number.

Dataset Files: product_batch1.csv (15), product_batch2.csv (7), product_batch3.csv (2) — from Product_SCD_Dynamic folder

Dataset Preview — product_batch1.csv (first 5 rows)

PRODUCT_ID	PRODUCT_NAME	CATEGORY	LIST_PRICE	SUPPLIER	UPDATED_AT
5101	Wireless Keyboard	Accessories	2499.00	TechSource	2026-09-01 09:00:00
5102	27-inch Monitor	Electronics	18999.00	VisionTech	2026-09-01 09:05:00
5103	USB-C Hub	Accessories	3499.00	ConnectPro	2026-09-01 09:10:00
5104	Noise Cancelling Headphones	Audio	12999.00	SoundWave	2026-09-01 09:15:00
5110	Business Laptop 14	Computers	74999.00	ComputeHub	2026-09-01 09:45:00

Batch 2: 5 products updated (PRODUCT_ID 5102, 5104, 5110, 5113, 5115) + 2 new products. Batch 3: Price cuts for 5102 and 5110.

Step 1 – Create Environment

Create: Database **PRODUCT_SCD_DB**, Schema **RAW**, CSV Format **CSV_FMT**, External Stage **S3_PROD_STAGE**

Create source table PRODUCT_RAW with: PRODUCT_ID, PRODUCT_NAME, CATEGORY, LIST_PRICE, SUPPLIER, UPDATED_AT

Step 2 – Create Snowpipe: PRODUCT_PIPE

Configure AUTO_INGEST=TRUE loading files from S3_PROD_STAGE into PRODUCT_RAW.

Step 3 – Ingest Batches & Create SCD Type 2 Dynamic Table

Load all three batches. Create **DT_PRODUCT_SCD2** with TARGET_LAG='1 minute'.

The Dynamic Table must maintain every version of each product.

Required columns:

- PRODUCT_ID, PRODUCT_NAME, CATEGORY, LIST_PRICE, SUPPLIER
- EFFECTIVE_FROM, EFFECTIVE_TO, IS_CURRENT (TRUE/FALSE), VERSION_NO

Use window logic based on PRODUCT_ID + UPDATED_AT.

Latest record must have IS_CURRENT=TRUE and EFFECTIVE_TO=NULL (or agreed future date).

Expected Row Counts:

- PRODUCT_RAW = 24 rows | DT_PRODUCT_SCD2 = 24 rows (full history)

Step 4 – Validate Key Records

Verify PRODUCT_ID = 5102 (27-inch Monitor) — must show 3 versions:

- Version 1: LIST_PRICE=18999, SUPPLIER=VisionTech, IS_CURRENT=FALSE
- Version 2: LIST_PRICE=17999, SUPPLIER=VisionTech Plus, IS_CURRENT=FALSE
- Version 3: LIST_PRICE=16999, SUPPLIER=VisionTech Plus, IS_CURRENT=TRUE

Verify PRODUCT_ID = 5110 (Business Laptop 14) — must also show 3 versions with decreasing price.

Tricky Element:

SCD Type 2 requires LAG() / LEAD() window functions to build EFFECTIVE_FROM and EFFECTIVE_TO. VERSION_NO must increment per product. Products with only one batch entry should have VERSION_NO=1, IS_CURRENT=TRUE, EFFECTIVE_TO=NULL. New products in batch 2 (5116, 5117) must appear with a single version.

Submission Requirements

1. Snowpipe Ingestion – PRODUCT_RAW with 24 rows.
2. SCD Type 2 History – DT_PRODUCT_SCD2 showing 3 versions each for 5102 and 5110.
3. Dynamic Table Refresh Confirmation.

Evaluation Rubric

#	Criterion	Marks
1	Environment & Snowpipe – All objects created, 24 raw rows ingested correctly	6
2	SCD Type 2 Dynamic Table – Full history with EFFECTIVE_FROM/TO, IS_CURRENT, VERSION_NO	8
3	Key Record Validation – 5102 and 5110 each show 3 correct versions	4
4	Dynamic Table Refresh & Pipeline State – Confirmed refresh, pipe paused	2
	TOTAL	**20**

QUESTION 6 – MEDIUM – SNOWPARK + STREAMLIT (FOOD ORDERS)

Marks: 20	Difficulty: Medium

Objective

Load food order data, perform SQL transformations using Snowpark Python (session.sql), persist the transformed outputs as Snowflake tables, and build an interactive Streamlit in Snowflake application with filters, KPI metrics, and charts.

Dataset File: food_orders.csv (50 rows) — from MEDIUM folder

Dataset Preview (first 4 rows)

DER_ID	ORDER_DATE	CUSTOMER_NAME	RESTAURANT	CUISINE	CITY	REGION	ORDER_AMT	DISC_AMT	DEL_FEE	DEL_MIN	PAY_MODE	STATUS	RATIN
9001	2026-01-04	Aarav Sharma	Spice Route	South Indian	Chennai	South	840.00	40.00	20.00	48	UPI	DELIVERED	4
9002	2026-02-06	Meera Iyer	Spice Route	South Indian	Chennai	South	930.00	50.00	30.00	52	Card	DELIVERED	3
9007	2026-03-10	Aditya Jain	Curry House	North Indian	Bengaluru	South	950.00	40.00	20.00	48	Cash	DELIVERED	4
9041	2026-01-10	Aarav Sharma	Spice Route	South Indian	Chennai	South	650.00	30.00	20.00	25	UPI	CANCELLED	3

Step 1 – Create Environment & Load Data

Create: Database **FOOD_ANALYTICS_DB**, Schema **ANALYTICS**, CSV Format **CSV_FMT**, Internal Stage **FOOD_STAGE**

Create table FOOD_ORDERS with all 14 columns above. Load food_orders.csv. Expected: 50 rows.

Step 2 – Snowpark SQL Transformations (using session.sql)

T1 – ORDERS_CURATED: Keep only DELIVERED orders. Add:

- NET_REVENUE = ORDER_AMOUNT - DISCOUNT_AMT + DELIVERY_FEE
- STAY_MONTH = DATE_TRUNC('MONTH', ORDER_DATE)

Expected: 40 DELIVERED rows. Validate total NET_REVENUE.

T2 – TOP_RESTAURANT_PER_CITY: Group by CITY + RESTAURANT. Calculate TOTAL_NET_REVENUE, ORDER_COUNT, AVG_RATING. Use RANK() with QUALIFY to retain only the top restaurant per city by TOTAL_NET_REVENUE.

Expected: 7 rows (one per city).

T3 – MONTHLY_CITY_TREND: Group by CITY + STAY_MONTH. Calculate TOTAL_NET_REVENUE, ORDER_COUNT. Use LAG() for PREV_MONTH_REVENUE and MOM_GROWTH_PCT.

Expected: Multiple rows. First month per city must show NULL for MOM_GROWTH_PCT.

T4 – CUISINE_SUMMARY: Group by CUISINE + CITY. Calculate TOTAL_NET_REVENUE, ORDER_COUNT, AVG_DELIVERY_MINUTES, AVG_RATING.

Expected: Multiple rows.

Step 3 – Build Streamlit App: FOOD_ORDERS_APP

Filters: CITY (st.multiselect, default all), CUISINE (st.multiselect), RATING (st.slider 1–5)

KPI Metrics (st.metric): Total Net Revenue, Total Orders, Average Delivery Time

Visuals: Bar chart – NET_REVENUE by CUISINE | Line chart – monthly revenue trend (MONTHLY_CITY_TREND)

Display TOP_RESTAURANT_PER_CITY as a table. All filters must update KPIs, charts, and filtered data.

Tricky Element:

The dataset includes CANCELLED, PENDING, and DELIVERED orders. Transformation 1 must correctly filter only DELIVERED. The MONTHLY_CITY_TREND must handle cities with single-month data showing NULL MOM_GROWTH_PCT. The CUISINE filter in Streamlit must also affect the delivery-time KPI since different cuisines have different delivery patterns.

Submission Requirements

1. Snowpark Transformation Results – All 4 tables with required columns and expected counts.
2. Streamlit Filters & KPI Validation – FOOD_ORDERS_APP with all filters working.
3. Streamlit Final Dashboard – Charts and TOP_RESTAURANT table visible.

Evaluation Rubric

#	Criterion	Marks
1	Environment & T1 ORDERS_CURATED – 40 DELIVERED rows, NET_REVENUE and STAY_MONTH correct	5
2	SQL Analytical Transformations – T2 ranking, T3 LAG/MOM, T4 cuisine aggregation correct	6
3	Streamlit Filters & KPI Metrics – CITY, CUISINE, RATING filters working with KPIs	4
4	Streamlit Visuals & Final Output – Charts, TOP_RESTAURANT table, filtered dashboard	5
	TOTAL	**20**

QUESTION 7 – MEDIUM – SNOWPARK DATAFRAME TRANSFORMATIONS + STREAMLIT (MOVIE BOOKINGS)

Marks: 20	Difficulty: Medium

Objective

Load movie booking data, perform five transformations using ONLY the Snowpark DataFrame API (no session.sql), persist the outputs as Snowflake tables, and build an interactive Streamlit in Snowflake dashboard.

Dataset File: movie_bookings.csv (50 rows) — from MEDIUM folder

Dataset Preview (first 3 rows)

ING_ID	SHOW_DATE	MOVIE	GENRE	LANGUAGE	THEATRE	CITY	REGION	SCREEN_FORMAT	TICKETS	TICKET_PRICE	CONV_FEE	CHANNEL	STA
01	2026-01-03	Sky Warriors	Action	English	Grand Cineplex	Chennai	South	IMAX	3	220.00	20.00	App	BOO
02	2026-01-04	Love Junction	Romance	Hindi	Metro Movies	Bengaluru	South	3D	2	250.00	30.00	Web	BOO
03	2026-01-05	Quantum Edge	Sci-Fi	English	Star Screens	Delhi	North	IMAX	4	230.00	25.00	App	BOO

Step 1 – Create Environment & Load Data

Create: Database **CINEMA_ANALYTICS_DB**, Schema **ANALYTICS**, CSV Format **CSV_FMT**, Internal Stage **MOVIE_STAGE**

Create table MOVIE_BOOKINGS with all 14 columns. Load movie_bookings.csv. Expected: 50 rows.

Step 2 – Snowpark DataFrame Transformations (NO session.sql)

Important: All five transformations must use Snowpark DataFrame API (filter, with_column, group_by, agg, sort). Save each using write.save_as_table(..., mode='overwrite').

T1 – BOOKINGS_CURATED: Keep only BOOKING_STATUS='BOOKED'. Add:

- GROSS_AMOUNT = TICKETS × TICKET_PRICE
- NET_REVENUE = GROSS_AMOUNT + CONVENIENCE_FEE
- SHOW_MONTH = month-level truncation of SHOW_DATE

Expected: 40 rows. Validate total NET_REVENUE.

T2 – GENRE_SUMMARY: Group by GENRE. Calculate TOTAL_NET_REVENUE, TOTAL_TICKETS, BOOKING_COUNT, AVG_NET_REVENUE. Expected: Multiple rows.

T3 – CITY_THEATRE_PERFORMANCE: Group by CITY + THEATRE. Calculate TOTAL_NET_REVENUE, BOOKING_COUNT. Keep BOOKING_COUNT >= 3. Sort by TOTAL_NET_REVENUE descending.

T4 – MONTHLY_GENRE_TREND: Group by SHOW_MONTH + GENRE. Calculate TOTAL_NET_REVENUE, TOTAL_TICKETS. Sort by SHOW_MONTH.

T5 – SCREEN_FORMAT_ANALYSIS: Group by SCREEN_FORMAT + LANGUAGE. Calculate BOOKING_COUNT, TOTAL_NET_REVENUE, AVG_TICKET_PRICE. Sort by TOTAL_NET_REVENUE descending.

Step 3 – Build Streamlit App: CINEMA_DASHBOARD

Filters: GENRE (st.selectbox with 'All'), REGION (st.multiselect), SCREEN_FORMAT (st.multiselect)

KPI Metrics: Total Net Revenue, Total Bookings, Average Tickets Per Booking

Charts: Bar chart – NET_REVENUE by CITY | Line chart – monthly NET_REVENUE

Display CITY_THEATRE_PERFORMANCE as a table. Filtered rows via st.dataframe.

Tricky Element:

SCREEN_FORMAT_ANALYSIS groups by both SCREEN_FORMAT and LANGUAGE — candidates must not group by SCREEN_FORMAT alone. T3 requires filtering AFTER aggregation (BOOKING_COUNT >= 3), not before. The SHOW_MONTH derivation must use date_trunc via Snowpark functions, not SQL strings.

Evaluation Rubric

#	Criterion	Marks
1	Environment & T1 BOOKINGS_CURATED – 40 BOOKED rows, GROSS_AMOUNT, NET_REVENUE, SHOW_MONTH correct	4
2	DataFrame Aggregations – T2, T3, T4, T5 using Snowpark API with correct outputs	8
3	Streamlit Filters – GENRE, REGION, SCREEN_FORMAT filters working via DataFrame operations	3
4	Streamlit KPI Metrics – All 3 KPIs respond to filter selections	2
5	Streamlit Charts & Interactivity – City bar chart, monthly line chart, table, filtered data	3
	TOTAL	**20**

QUESTION 8 – MEDIUM – SNOWPARK DATAFRAME TRANSFORMATIONS + STREAMLIT (RIDE TRIPS)

Marks: 20	Difficulty: Medium

Objective

Load ride-hailing trip data, perform five transformations using ONLY the Snowpark DataFrame API (no session.sql), save the results as Snowflake tables, and build an interactive Streamlit in Snowflake dashboard for operational analytics.

Dataset File: ride_trips.csv (50 rows) — from MEDIUM folder

Dataset Preview (first 3 rows)

TRIP_ID	TRIP_DATE	DRIVER_ID	CITY	REGION	VEHICLE_TYPE	DIST_KM	DUR_MIN	BASE_FARE	SURGE_MULT	DISC_AMT	PAY_MODE	TRIP_STATUS	RIDER_RATIN
10001	2026-01-02	D001	Chennai	South	Mini	4.5	18	180.00	1.5	0.00	UPI	COMPLETED	5
10002	2026-01-03	D002	Bengaluru	South	Sedan	6.2	24	220.00	1.2	20.00	Card	COMPLETED	4
10003	2026-01-04	D003	Delhi	North	SUV	8.1	32	310.00	1.0	0.00	Cash	COMPLETED	3

Step 1 – Create Environment & Load Data

Create: Database **RIDE_ANALYTICS_DB**, Schema **ANALYTICS**, CSV Format **CSV_FMT**, Internal Stage **RIDE_STAGE**

Create table RIDE_TRIPS with all 14 columns. Load ride_trips.csv. Expected: 50 rows.

Step 2 – Snowpark DataFrame Transformations (NO session.sql)

Important: Use only Snowpark DataFrame API. Save each via write.save_as_table(..., mode='overwrite').

T1 – TRIPS_CURATED: Keep only TRIP_STATUS='COMPLETED'. Add:

- FINAL_FARE = (BASE_FARE × SURGE_MULTIPLIER) - DISCOUNT_AMT
- FARE_PER_KM = FINAL_FARE / DISTANCE_KM (round to 2 decimal places)
- TRIP_MONTH = month-level truncation of TRIP_DATE

Expected: 40 rows. Validate total FINAL_FARE.

T2 – CITY_PERFORMANCE: Group by CITY + REGION. Calculate TOTAL_FINAL_FARE, TOTAL_TRIPS, TOTAL_DISTANCE, AVG_RIDER_RATING. Sort by TOTAL_FINAL_FARE descending.

T3 – DRIVER_SUMMARY: Group by DRIVER_ID + CITY. Calculate TOTAL_TRIPS, TOTAL_FARE, AVG_FARE_PER_KM. Keep TOTAL_TRIPS >= 2. Sort by TOTAL_FARE descending.

T4 – MONTHLY_VEHICLE_TREND: Group by TRIP_MONTH + VEHICLE_TYPE. Calculate TOTAL_FINAL_FARE, TOTAL_TRIPS. Sort by TRIP_MONTH, VEHICLE_TYPE.

T5 – SURGE_ANALYSIS: From TRIPS_CURATED, group by VEHICLE_TYPE + CITY. Calculate AVG_SURGE_MULTIPLIER, HIGH_SURGE_COUNT (SURGE_MULTIPLIER > 1.2), TOTAL_TRIPS. Sort by AVG_SURGE_MULTIPLIER descending.

Step 3 – Build Streamlit App: RIDE_ANALYTICS_APP

Filters: CITY (st.multiselect), VEHICLE_TYPE (st.selectbox with 'All'), RIDER_RATING (st.slider 1–5)

KPI Metrics: Total Final Fare, Total Trips, Average Fare Per KM

Charts: Bar chart – TOTAL_FINAL_FARE by CITY | Line chart – monthly fare trend by VEHICLE_TYPE

Display DRIVER_SUMMARY as a table. Filtered rows via st.dataframe.

Tricky Element:

FARE_PER_KM must handle potential division-edge cases. SURGE_ANALYSIS uses conditional aggregation: HIGH_SURGE_COUNT = count where SURGE_MULTIPLIER > 1.2, implemented with Snowpark when() / otherwise(). T3's DRIVER_SUMMARY filters must be applied AFTER grouping. The monthly line chart should show separate lines per VEHICLE_TYPE.

Evaluation Rubric

#	Criterion	Marks
1	Environment & T1 TRIPS_CURATED – 40 COMPLETED rows, FINAL_FARE, FARE_PER_KM, TRIP_MONTH correct	4
2	DataFrame Aggregations – T2, T3, T4, T5 using Snowpark API with correct outputs	8
3	Streamlit Filters – CITY, VEHICLE_TYPE, RIDER_RATING filters working	3
4	Streamlit KPI Metrics – All 3 KPIs respond correctly to filters	2
5	Streamlit Charts & Interactivity – City bar chart, monthly line chart per vehicle type, driver table	3
	TOTAL	**20**

■■ SECTION: EXTRA (with Datasets) ■■

★ *The following two extra questions include their own datasets directly within this question paper.*

QUESTION 9 – EXTRA – SEMI-STRUCTURED DATA (JSON) WITH LATERAL FLATTEN

Marks: Bonus	Difficulty: Extra

Objective

Load nested JSON e-commerce order data into a VARIANT column, extract required fields using dot notation, flatten the nested items array using LATERAL FLATTEN, save the flattened result as a table, and perform category-level sales analysis.

Dataset — Provided Below (online_orders.json)

Copy this JSON content, save it as online_orders.json and upload it to Snowflake.


```
[ { "order_id": "ORD-3001", "order_date": "2026-09-01", "customer": { "name": "Aarav Sharma",
"city": "Chennai", "contact": { "email": "aarav.sharma@mail.com", "phone": "9350596752" } },
"payment": { "mode": "UPI", "status": "PAID" }, "items": [ {"sku": "SKU-101","name": "Wireless
Mouse","category": "Accessories","qty": 2,"price": 999}, {"sku": "SKU-205","name": "USB-C
Hub","category": "Accessories","qty": 1,"price": 1899} ] }, { "order_id": "ORD-3002",
"order_date": "2026-09-02", "customer": { "name": "Meera Iyer", "city": "Bengaluru", "contact":
{ "email": "meera.iyer@mail.com", "phone": "9845123478" } }, "payment": { "mode": "Card",
"status": "PAID" }, "items": [ {"sku": "SKU-305","name": "Noise Cancelling
Headphones","category": "Audio","qty": 1,"price": 12999}, {"sku": "SKU-406","name": "Bluetooth
Speaker","category": "Audio","qty": 2,"price": 4999} ] }, { "order_id": "ORD-3003",
"order_date": "2026-09-03", "customer": { "name": "Rohan Mehta", "city": "Mumbai", "contact": {
"email": "rohan.mehta@mail.com", "phone": "9765432109" } }, "payment": { "mode": "Cash",
"status": "PENDING" }, "items": [ {"sku": "SKU-102","name": "Mechanical Keyboard","category":
"Accessories","qty": 1,"price": 2799}, {"sku": "SKU-501","name": "27-inch Monitor","category":
"Electronics","qty": 1,"price": 16999} ] }, { "order_id": "ORD-3004", "order_date":
"2026-09-04", "customer": { "name": "Kavya Rao", "city": "Hyderabad", "contact": { "email":
"kavya.rao@mail.com", "phone": "9988776655" } }, "payment": { "mode": "UPI", "status": "PAID" },
"items": [ {"sku": "SKU-601","name": "Office Chair","category": "Furniture","qty": 1,"price":
7499}, {"sku": "SKU-602","name": "Study Desk","category": "Furniture","qty": 1,"price": 9499},
{"sku": "SKU-103","name": "Laptop Stand","category": "Accessories","qty": 1,"price": 1499} ] },
{ "order_id": "ORD-3005", "order_date": "2026-09-05", "customer": { "name": "Arjun Nair",
"city": "Kochi", "contact": { "email": "arjun.nair@mail.com", "phone": "9876543210" } },
"payment": { "mode": "Wallet", "status": "PAID" }, "items": [ {"sku": "SKU-306","name":
"Wireless Earbuds","category": "Audio","qty": 2,"price": 2599}, {"sku": "SKU-502","name":
"Webcam Pro","category": "Electronics","qty": 1,"price": 3299} ] }, { "order_id": "ORD-3006",
"order_date": "2026-09-06", "customer": { "name": "Neha Verma", "city": "Delhi", "contact": {
"email": "neha.verma@mail.com", "phone": "9812345678" } }, "payment": { "mode": "Card",
"status": "PAID" }, "items": [ {"sku": "SKU-503","name": "4K Monitor","category":
"Electronics","qty": 1,"price": 30999}, {"sku": "SKU-603","name": "Bookshelf","category":
"Furniture","qty": 1,"price": 5799} ] }, { "order_id": "ORD-3007", "order_date": "2026-09-07",
"customer": { "name": "Aditya Jain", "city": "Pune", "contact": { "email":
"aditya.jain@mail.com", "phone": "9898987654" } }, "payment": { "mode": "UPI", "status": "PAID"
}, "items": [ {"sku": "SKU-307","name": "Soundbar","category": "Audio","qty": 1,"price": 9999},
{"sku": "SKU-104","name": "USB Hub","category": "Accessories","qty": 3,"price": 1499} ] }, {
"order_id": "ORD-3008", "order_date": "2026-09-08", "customer": { "name": "Priya Menon", "city":
"Chennai", "contact": { "email": "priya.menon@mail.com", "phone": "9700012345" } }, "payment": {
"mode": "Card", "status": "PAID" }, "items": [ {"sku": "SKU-504","name": "Portable SSD
1TB","category": "Electronics","qty": 1,"price": 8999}, {"sku": "SKU-604","name": "LED Desk
Lamp","category": "Furniture","qty": 2,"price": 1399} ] }, { "order_id": "ORD-3009",
"order_date": "2026-09-09", "customer": { "name": "Vikram Singh", "city": "Jaipur", "contact": {
"email": "vikram.singh@mail.com", "phone": "9922334455" } }, "payment": { "mode": "Cash",
"status": "PENDING" }, "items": [ {"sku": "SKU-105","name": "Power Bank 20000mAh","category":
"Accessories","qty": 2,"price": 2199}, {"sku": "SKU-308","name": "Bluetooth Speaker","category":
```

```
"Audio","qty": 1,"price": 3499} ] }, { "order_id": "ORD-3010", "order_date": "2026-09-10",
"customer": { "name": "Ananya Das", "city": "Kolkata", "contact": { "email":
"ananya.das@mail.com", "phone": "9797971234" } }, "payment": { "mode": "UPI", "status": "PAID"
}, "items": [ {"sku": "SKU-505","name": "Gaming Laptop","category": "Electronics","qty":
1,"price": 89999} ] }, { "order_id": "ORD-3011", "order_date": "2026-09-11", "customer": {
"name": "Siddharth Rao", "city": "Bengaluru", "contact": { "email": "siddharth.rao@mail.com",
"phone": "9685741236" } }, "payment": { "mode": "Wallet", "status": "PAID" }, "items": [ {"sku":
"SKU-106","name": "Wireless Keyboard","category": "Accessories","qty": 1,"price": 2499}, {"sku":
"SKU-309","name": "Noise Cancelling Earbuds","category": "Audio","qty": 2,"price": 4999},
{"sku": "SKU-605","name": "Ergonomic Chair","category": "Furniture","qty": 1,"price": 12999} ]
}, { "order_id": "ORD-3012", "order_date": "2026-09-12", "customer": { "name": "Divya Nair",
"city": "Kochi", "contact": { "email": "divya.nair@mail.com", "phone": "9654321876" } },
"payment": { "mode": "Card", "status": "PAID" }, "items": [ {"sku": "SKU-506","name": "Smart TV
55-inch","category": "Electronics","qty": 1,"price": 54999}, {"sku": "SKU-107","name": "Wall
Mount Kit","category": "Accessories","qty": 1,"price": 1999} ] } ]
```


Step 1 – Create Environment

Create: Database **ECOM_JSON_DB**, Schema **RAW**

Create JSON Format: JSON_FMT (TYPE=JSON, STRIP_OUTER_ARRAY=TRUE)

Create Internal Stage: **JSON_STAGE**

Create Table: **ONLINE_ORDERS_RAW** with single column ORDER_DATA VARIANT

Step 2 – Load JSON Data

Upload online_orders.json to JSON_STAGE. Load using COPY INTO. Expected: 12 rows (one row per order).

Step 3 – Query Using Dot Notation

Using dot notation and casting, extract ORDER_ID, ORDER_DATE, CUSTOMER_NAME, CITY, EMAIL, PAYMENT_MODE, PAYMENT_STATUS from the VARIANT column.

Field paths: customer.name, customer.city, customer.contact.email, payment.mode, payment.status

Display only orders where PAYMENT_STATUS = 'PAID'. Expected: 10 orders (ORD-3003 and ORD-3009 are PENDING).

Step 4 – Flatten the Items Array

Use LATERAL FLATTEN on the items array to create one row per item.

Extract: ORDER_ID, CUSTOMER_NAME, SKU, ITEM_NAME, CATEGORY, QTY, PRICE

Calculate: LINE_TOTAL = QTY × PRICE

Save as: **ORDER_ITEMS_FLAT**. Expected: 27 rows.

Step 5 – Category Analysis

From ORDER_ITEMS_FLAT, calculate total QTY and total SALES per CATEGORY. Sort by total SALES descending.

Expected category totals:

- Electronics → HIGH sales (Gaming Laptop, Smart TV, 4K Monitor dominate)
- Audio → Multiple units of earbuds and speakers contribute significantly
- Furniture → Chair + Desk + Lamp orders across multiple cities
- Accessories → High quantity but lower unit prices

Tricky Element:

The JSON has 3-level nesting for email (customer.contact.email) — a two-level dot path will fail. STRIP_OUTER_ARRAY must be TRUE or each order appears as a nested array. LATERAL FLATTEN produces one row per item in the items array; ORDER_ITEMS_FLAT row count depends on total items across all 12 orders.

Submission Requirements

1. Data Load – ONLINE_ORDERS_RAW with 12 rows.
2. Dot Notation Output – 10 PAID orders with correct nested fields.

3. Flatten & Category Summary – ORDER_ITEMS_FLAT and category analysis.

Evaluation Rubric

#	Criterion	Marks
1	JSON Setup & Data Load – VARIANT table loaded with correct row count	2
2	Dot Notation Extraction – 3-level nesting (email) and PAID filter correct	2
3	LATERAL FLATTEN – ORDER_ITEMS_FLAT created with all item-level fields and LINE_TOTAL	4
4	Category Analysis – Correct aggregation from flattened table, sorted by sales	2
	TOTAL	**10**

QUESTION 10 – EXTRA – ROLE BASED ACCESS CONTROL, MASKING & ROW ACCESS POLICY

Marks: Bonus	Difficulty: Extra

Objective

Load employee data, create role-based access with a hierarchy, protect sensitive columns using masking policies, and restrict employee rows by department using a Row Access Policy.

Dataset — Provided Below (employee_secure.csv)

Save the data below as employee_secure.csv and load it into Snowflake.

EMP_ID	EMP_NAME	DEPARTMENT	DESIGNATION	CITY	REGION	EMAIL	PHONE_NUMBER	SALARY	JOIN_DATE
2201	Aarav Sharma	Engineering	Senior Engineer	Chennai	South	aarav.sharma@corp.com	9350596752	1240000.00	2020-01-15
2202	Meera Iyer	HR	HR Manager	Bengaluru	South	meera.iyer@corp.com	9845123478	980000.00	2021-03-10
2203	Rohan Mehta	Finance	Finance Analyst	Mumbai	West	rohan.mehta@corp.com	9765432109	1120000.00	2019-07-22
2204	Kavya Rao	Engineering	Data Engineer	Hyderabad	South	kavya.rao@corp.com	9988776655	1180000.00	2022-02-14
2205	Arjun Nair	Sales	Sales Manager	Kochi	South	arjun.nair@corp.com	9876543210	1050000.00	2020-11-05
2206	Neha Verma	Operations	Operations Lead	Delhi	North	neha.verma@corp.com	9812345678	1010000.00	2021-06-18
2207	Aditya Jain	Engineering	Cloud Engineer	Pune	West	aditya.jain@corp.com	9898987654	1260000.00	2018-09-27
2208	Priya Menon	Marketing	Marketing Specialist	Chennai	South	priya.menon@corp.com	9700012345	920000.00	2023-01-09
2209	Vikram Singh	Finance	Senior Accountant	Jaipur	North	vikram.singh@corp.com	9922334455	1090000.00	2019-12-02
2210	Ananya Das	Engineering	BI Developer	Kolkata	East	ananya.das@corp.com	9797971234	1150000.00	2022-05-16
2211	Siddharth Rao	Engineering	Software Engineer	Bengaluru	South	siddharth.rao@corp.com	9685741236	1080000.00	2021-08-11
2212	Divya Nair	HR	Recruiter	Kochi	South	divya.nair@corp.com	9654321876	860000.00	2023-04-03
2213	Manish Shah	Sales	Account Executive	Ahmedabad	West	manish.shah@corp.com	9543216789	940000.00	2020-10-19
2214	Pooja Kulkarni	Operations	Process Analyst	Pune	West	pooja.kulkarni@corp.com	9432167890	900000.00	2022-07-07
2215	Rahul Menon	Engineering	DevOps Engineer	Hyderabad	South	rahul.menon@corp.com	9321654780	1210000.00	2019-04-29
2216	Sneha Pillai	Marketing	Campaign Manager	Kochi	South	sneha.pillai@corp.com	9216543870	970000.00	2021-12-13
2217	Aditi Sharma	Finance	Financial Controller	Delhi	North	aditi.sharma@corp.com	9109876543	1320000.00	2018-06-25
2218	Varun Mehta	Engineering	Solution Architect	Mumbai	West	varun.mehta@corp.com	9098765432	1450000.00	2017-03-08
2219	Ritika Ghosh	HR	HR Executive	Kolkata	East	ritika.ghosh@corp.com	9087654321	840000.00	2023-09-21
2220	Nitin Verma	Operations	Operations Analyst	Jaipur	North	nitin.verma@corp.com	9076543218	890000.00	2022-11-30

Step 1 – Create Environment & Load Data

Create: Database **SECURE_HR_DB**, Schema **CORE**

Create Table EMPLOYEE_MASTER with: EMP_ID NUMBER, EMP_NAME STRING, DEPARTMENT STRING, DESIGNATION STRING, CITY STRING, REGION STRING, EMAIL STRING, PHONE_NUMBER STRING, SALARY NUMBER(12,2), JOIN_DATE DATE

Load employee_secure.csv using COPY INTO. Expected: 20 rows.

Step 2 – Create Role Hierarchy

Create roles: **HR_ADMIN** and **HR_ANALYST**

HR_ADMIN: USAGE on DB + Schema, SELECT + INSERT + UPDATE + DELETE on EMPLOYEE_MASTER

HR_ANALYST: USAGE on DB + Schema, SELECT on EMPLOYEE_MASTER only

Role hierarchy: Grant HR_ANALYST to HR_ADMIN. Grant HR_ADMIN to SYSADMIN.

Grant warehouse access and assign both roles to your user.

Step 3 – Apply Masking Policies

EMAIL_MASK: HR_ADMIN → actual EMAIL | Others → '***@corp.com' | Apply to EMAIL column

SALARY_MASK: HR_ADMIN → actual SALARY | Others → NULL | Apply to SALARY column

PHONE_MASK: HR_ADMIN → actual PHONE_NUMBER | Others → 'XXXX-XXXX-XX' | Apply to PHONE_NUMBER column

Step 4 – Apply Row Access Policy

Create table **DEPT_ROLE_MAP** with columns ROLE_NAME and DEPARTMENT

Add the following mappings:

- HR_ANALYST → Engineering | HR_ANALYST → Sales | HR_ANALYST → Marketing

Create Row Access Policy: **DEPT_RAP** and apply it to the DEPARTMENT column of EMPLOYEE_MASTER.

Expected access: HR_ADMIN → all 20 rows | HR_ANALYST → Engineering (6) + Sales (2) + Marketing (2) = 10 rows

Step 5 – Test the Security

HR_ADMIN: Query EMPLOYEE_MASTER → expect 20 rows, actual EMAIL, actual SALARY, actual PHONE_NUMBER

HR_ANALYST: Query EMPLOYEE_MASTER → expect 10 rows (Engineering + Sales + Marketing only), EMAIL masked, SALARY=NULL, PHONE masked

Attempt UPDATE using HR_ANALYST — should fail with permission denied.

Tricky Element:

Three masking policies must be created and applied — EMAIL, SALARY, and PHONE. The Row Access Policy filters by DEPARTMENT (not REGION as in previous assessments). HR_ANALYST sees 10 rows across 3 departments: Engineering (6 employees: 2201, 2204, 2207, 2210, 2211, 2215), Sales (2 employees: 2205, 2213), Marketing (2 employees: 2208, 2216). The DEPT_ROLE_MAP must have exactly 3 entries for HR_ANALYST.

Submission Requirements

1. Implementation Screenshot – Role creation, masking policies, DEPT_ROLE_MAP, Row Access Policy, attachments.
2. HR_ADMIN Validation – 20 rows visible, EMAIL/SALARY/PHONE all unmasked.
3. HR_ANALYST Validation – Only 10 rows (correct departments), EMAIL masked, SALARY=NULL, PHONE masked.

Evaluation Rubric

#	Criterion	Marks
1	Environment & Data Load – EMPLOYEE_MASTER created with 20 rows	3
2	RBAC Configuration – HR_ADMIN (full DML) and HR_ANALYST (SELECT only) with hierarchy	5
3	Masking Policies – EMAIL, SALARY, PHONE_NUMBER protected correctly per role	5
4	Row Access Policy – DEPT_RAP restricts HR_ANALYST to Engineering + Sales + Marketing (10 rows)	4
5	Final Role Validation – HR_ADMIN sees 20 rows unmasked; HR_ANALYST sees 10 rows masked	3
	TOTAL	**20**

★ END OF QUESTION PAPER ★

Total Core Questions: 8 (Q1–Q8) | Total with Extra: 10 | Core Marks: 140